DBMS Detailed Study Notes

File Operations, Indexes, & Transactions

Comprehensive Exam Preparation Material

May 2026

Table of Contents

1. Operations on Files (Section 16.5)

File operations in a Database Management System (DBMS) are the fundamental commands used to access, manage, and modify data stored on disk. These operations are broadly grouped into two categories: retrieval (read-only) and update (modify data).

1.1 Two Main Categories of File Operations

> **Retrieval Operations**
>
> Purpose: To locate specific records so their field values can be examined or processed. Effect: They do not change any data in the file (Read-only).

> **Update Operations**
>
> Purpose: To modify the file's contents. Effect: They change the file by inserting new records, deleting existing records, or modifying specific field values within a record.

1.2 Selection Conditions (Filtering Conditions)

Whenever we retrieve, delete, or modify a record, we usually do not want to affect the entire database. We use **selection conditions** to specify criteria that the desired records must satisfy.

Table 1 ***Types of Selection Conditions***

Type	Description	Example
Simple	Equality comparison on a single field	(Ssn = '123456789') or (Department = 'Research')
Complex	Other comparison operators (>, <, ≥, ≤) or Boolean expressions with AND/OR	(Salary ≥ 30000) AND (Department = 'Research')

How DBMS Handles Complex Conditions: Disk searches are generally optimized for simple conditions. If you give the DBMS a complex condition, it decomposes it to extract a

simple condition first.

Example Process: If the condition is `((Salary ≥ 30000) AND (Department = 'Research'))`, the DBMS might use the simple condition `(Department = 'Research')` to physically locate those specific records on the disk. Once located and brought into memory, each record is then checked against the second part of the condition `(Salary ≥ 30000)` to see if it is a full match.

> **The "Current Record" Concept**
>
> When a search condition matches multiple records, the DBMS locates the first record (based on its physical sequence on the disk). This record is designated as the **current record**. Any subsequent search operations (like finding the next match) will commence from this exact position.

1.3 Standard File Operations

Different systems use different commands, but here is the representative set of standard operations used by high-level programs (like DBMS software) to access records.

Record-at-a-Time Operations

These operations apply to a single record at a time.

Table 2 ***Record-at-a-Time File Operations***

Operation	Description
Open	Prepares the file for reading/writing. Allocates main memory buffers (typically at least two) to hold data blocks from disk. Retrieves the file header. Sets the file pointer to the beginning.
Reset	Sets the file pointer of an already open file back to the very beginning.
Find (Locate)	Searches for the first record satisfying the search condition. Transfers the disk block into a buffer (if not already there). The file pointer now points to this record, making it the current record.
Read (Get)	Copies the current record from the memory buffer into a program variable. Usually advances the current record pointer to the next record, which might trigger reading the next disk block.
FindNext	Searches for the next record satisfying the condition. Transfers its block into the buffer (if needed). The newly found record becomes the new current record.
Delete	Deletes the current record and eventually updates the file on physical disk.
Modify	Changes some field values of the current record and eventually updates the physical disk.
Insert	Locates the correct block for the new record, transfers that block to a buffer, writes the new record, and eventually writes the buffer back to disk.
Close	Completes file access by releasing allocated memory buffers and doing necessary cleanup.

> **Streamlined Operation: Scan**
>
> A hybrid operation that combines Find, FindNext, and Read. If the file is just opened/reset, Scan returns the first record. Otherwise, it returns the next record. It can also accept a search condition to only return matching records.

Set-at-a-Time Operations

These are higher-level operations that manipulate multiple records at once:

- **FindAll**: Locates all records satisfying a condition.
- **Find n (Locate n)**: Searches for the first record satisfying a condition, then continues to locate the next *n* records. Transfers blocks containing all located records into memory.
- **FindOrdered**: Retrieves all records in a specifically requested order.

- **Reorganize**: Starts a reorganization process. Some file setups require maintenance, like physically re-sorting all file records based on a specific field.

1.4 File Organization vs. Access Method

It is crucial to understand the difference between these two terms:

Table 3 ***File Organization vs. Access Method***

Concept	Definition	Example
File Organization	The **physical structure**. How data is organized into records, blocks, and access structures on the storage medium.	Ordering records sequentially by Name.
Access Method	The **logical software actions**. A group of operations (like Find, Read, Insert) applied to a file.	Indexed access method.

Rule of Thumb: You can apply several access methods to one file organization, but some access methods require a specific organization. For example, you cannot use an "indexed access method" if the file organization does not physically include an "index".

1.5 Design Choices in File Organization

Database Administrators (DBAs) must choose file organizations based on how the file is used.

Static vs. Dynamic Files

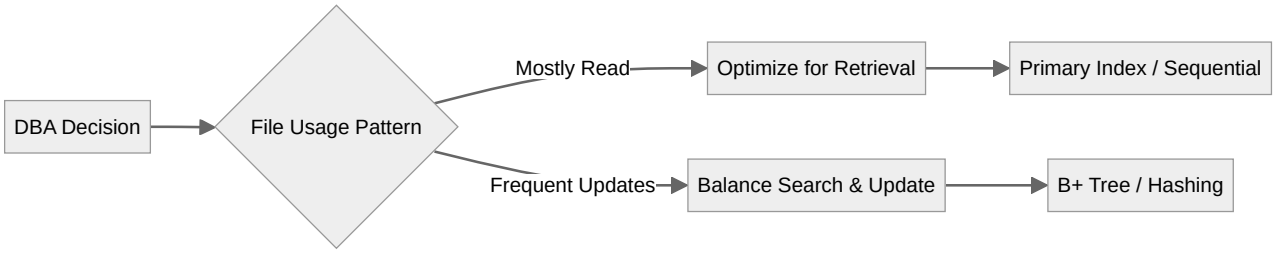
- **Static / Read-only files:** Rarely updated. Optimization focuses entirely on fast retrieval. (e.g., Data Warehouses predominantly use read-only files).
- **Dynamic files:** Frequently updated (insertions/deletions). Optimization must balance fast searching with efficient updating.

Conflicting Requirements (The DBA's Dilemma)

Imagine an EMPLOYEE file:

- **Application 1** constantly searches for employees by Ssn. The best organization is to physically order the file by Ssn or create an index on Ssn.

- **Application 2** needs to print paychecks grouped by Department. The best organization is to cluster records by Department.
- **The Conflict:** You can only physically sort a file one way on the disk. DBAs must make difficult design choices based on the expected mix and importance of different operations.



2. Primary Indexes (Section 17.1.1)

2.1 What is a Primary Index?

A primary index is a supplementary, ordered file that acts as an "access structure" to help you quickly search for and access data records in the main data file.

> **Key Characteristics of a Primary Index**
>
> - It is an **ordered file**.
> - Its records are of **fixed length**.
> - It consists of exactly two fields:
>
> 1. **Primary Key:** Has the same data type as the ordering key field of the main data file.
> 2. **Block Pointer:** A pointer to a physical disk block (block address).

2.2 How the Primary Index is Structured

In the data file shown in Figure 1, the main Data File is physically ordered by the Name field (making Name the primary key). The data file is divided into disk blocks.

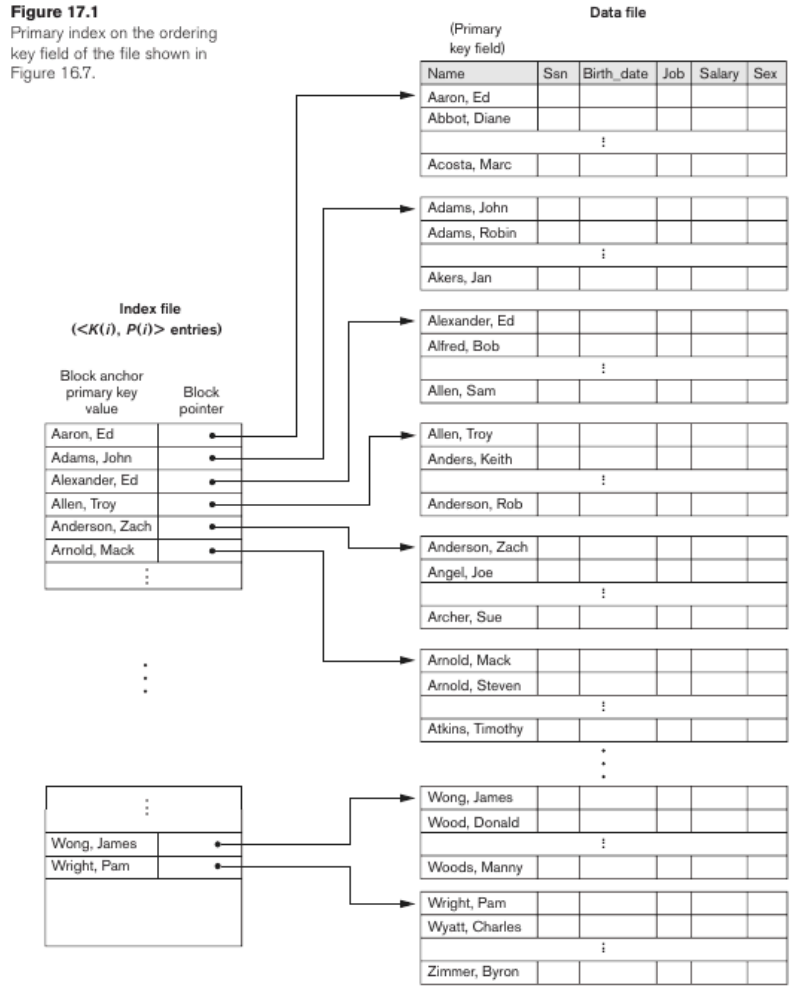


Figure 1 *Primary index on the ordering key field of the data file. The index file contains <K(i), P(i)> entries where K(i) is the block anchor and P(i) points to the data block.*

The **Index Entry format** is `<K(i), P(i)>`:

- There is exactly **one index entry per block** in the data file.
- **K(i)**: The value of the primary key for the **first record in block i**. This first record is critically called the **Block Anchor** (or anchor record).
- **P(i)**: A pointer to block *i*.

Data extraction from Figure 1:

- **Block 1** of Data File: The first record (Block Anchor) is "Aaron, Ed".
  Index Entry 1: `<K(1) = (Aaron, Ed), P(1) = address of block 1>`
- **Block 2** of Data File: The first record (Block Anchor) is "Adams, John".
  Index Entry 2: `<K(2) = (Adams, John), P(2) = address of block 2>`

Note on Pointers: The pointer *P(i)* in an entry can be a physical address, a record address (relative address within a file), or a logical address mapped to a physical one.

2.3 Dense vs. Sparse Indexes

Table 4 ***Dense vs. Sparse Index Comparison***

Property	Dense Index	Sparse (Nondense) Index
Index entries	One for **every single** search key value (every record)	One for **some** search values only
Space required	More space	Less space
Search speed	Directly points to record	Must search within block after locating it
Primary index?	No	Yes -- Primary index is **ALWAYS** sparse

> **Important Concept**
>
> A Primary Index is **ALWAYS** a Sparse Index. Why? Because it only contains an entry for the block anchors (one per block), not an entry for every single employee in the file.

2.4 Why are Primary Indexes Highly Efficient?

Because the index is small and ordered, we can perform a **Binary Search** on the index file, which requires drastically fewer disk accesses than searching the main data file.

Mathematical Efficiency Example

The Setup (Main Data File):

- Total records (r) = 30,000
- Block size (B) = 1024 bytes
- Record length (R) = 100 bytes
- Blocking factor (bfr) = $\lfloor B/R \rfloor$ = 10 records per block
- Total data blocks (b) = $\lceil r/bfr \rceil$ = 3000 blocks
- **Cost without Index:** Binary search on data file requires $\lceil \log_2 b \rceil$ = $\lceil \log_2 3000 \rceil$ = 12 block accesses.

The Improvement (With Primary Index):

- Key field size (V) = 9 bytes
- Pointer size (P) = 6 bytes
- Index entry size (R_i) = $V + P$ = 15 bytes
- Index blocking factor (bfr_i) = $\lfloor B/R_i \rfloor$ = 68 entries per block
- Total index entries (r_i) = b = 3000 (one for each data block)
- Total index blocks (b_i) = $\lceil r_i/bfr_i \rceil$ = 45 blocks
- **Cost with Index:** Binary search on the index requires $\lceil \log_2 b_i \rceil$ = 6 accesses. Plus 1 final access to grab the actual data block = **7 block accesses total**.

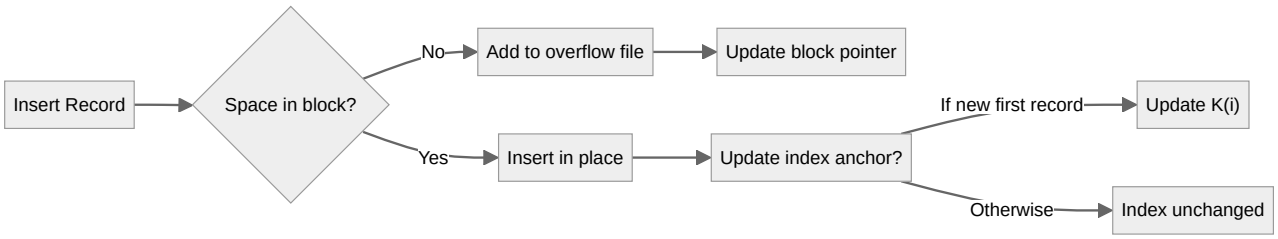
> **Summary**
>
> Without index: 12 block accesses. With primary index: 7 block accesses. The primary index nearly **doubles** search efficiency, and the gains are even more dramatic for larger files.

2.5 The Major Problem with Primary Indexes

Because the main file is **physically ordered**, insertions and deletions are very expensive. If you insert a new record, you have to physically move existing records down to make space. Moving records changes the Block Anchors, forcing continuous updates to the Primary Index.

Solutions:

- Use an **unordered overflow file** for new insertions.
- Use **linked lists** of overflow records per block.
- Use **deletion markers (tombstones)** instead of physically removing records.



3. Multilevel Indexes (Section 17.2)

3.1 Concept and Fan-out

Standard indexing schemes (like the Primary Index) use a binary search on the index file.

- A binary search reduces the search space by a factor of 2 at each step.
- Number of accesses required: $\lceil \log_2 b_i \rceil$ for b_i index blocks.

A **multilevel index** is designed to reduce the search space significantly faster than a binary search. Instead of dividing the search space by 2, it divides it by the index's blocking factor (bfr_i).

> **Fan-out (fo)**
> The blocking factor of the index (bfr_i) is referred to as the **fan-out** (fo). This represents how many n-ways the search space is divided at each step.

Math Comparison: Searching a multilevel index requires approximately $\lceil \log_{fo} b_i \rceil$ block accesses, as opposed to $\lceil \log_2 b_i \rceil$. Since the fan-out is usually much larger than 2, the number of accesses plummets.

Real-world Example of Fan-out Power

Assume a standard block size of **4096 bytes**.

- If a block pointer is **7 bytes** (which can accommodate 2^{56} blocks).
- If the search key (like an SSN) is **9 bytes**.
- The size of one index entry = $9 + 7$ = **16 bytes**.
- The Fan-out (fo) = $\lfloor 4096/16 \rfloor$ = **256** entries per block.
- **Result:** Instead of dividing the search space in half (2) at each step, a multilevel index divides it by **256** at each step!

3.2 B+ Trees

Multilevel indexes naturally lead to a structure called the **B+ Tree**, which is the most widely used index structure in modern DBMSs.

> **B+ Tree Properties**
>
> - All paths from root to leaf have the same length (balanced tree).
> - Each internal node has between $\lceil fo/2 \rceil$ and fo children.
> - Leaf nodes contain actual data pointers (or the data records themselves in a clustered index).
> - Leaf nodes are linked together for efficient range queries.

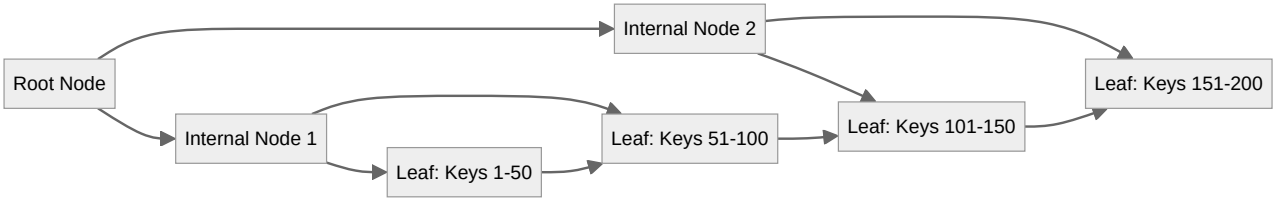
Table 5 ***B+ Tree Order (m) vs. Fan-out***

Parameter	Definition	Typical Value
Order (m)	Maximum number of pointers per node	100 - 500
Fan-out	Actual average number of pointers used	50 - 250
Height	Number of levels from root to leaf	2 - 4 for millions of records

Insertion in B+ Tree: Insertions may cause node splitting (if a node exceeds fo entries), which can propagate up to the root, increasing tree height. Because splitting is localized, insertions are much cheaper than reorganizing an entire sorted file.

Deletion in B+ Tree: Deletions may cause node underflow (below $\lceil fo/2 \rceil$ entries). Remedies: redistribution from a sibling or merging with a sibling.

> **Why B+ Trees Dominate**
>
> 1. They maintain automatic balance without expensive reorganization.
> 2. They support both exact-match and range queries efficiently.
> 3. Insertions and deletions are logarithmic time.
> 4. Leaf-level linking enables fast sequential access after a search.


Root Node
Internal Node 2
Internal Node 1
Leaf: Keys 1-50
Leaf: Keys 51-100
Leaf: Keys 101-150
Leaf: Keys 151-200

4. Secondary and Clustering Indexes (Section 17.1.2, 17.1.3)

These index types solve problems that primary indexes cannot. They allow efficient access by fields other than the primary ordering key.

4.1 Clustering Index

If records are **physically ordered** on a non-key field (a field that does not have a unique value for each record), the index on that field is called a **clustering index**.

> **Clustering Index Characteristics**
>
> - There is **one entry per distinct value** of the clustering field, not one per block.
> - Each entry points to the **first block** that contains a record with that value.
> - All records with the same clustering field value are stored **consecutively**.
> - It is a **sparse index** because there is one entry per distinct value, not per record.

Example: An EMPLOYEE file physically ordered by Department. The clustering index has one entry for "Research", pointing to the first block with Research employees. All Research employees are stored contiguously.

Table 6 ***Primary vs. Clustering Index***

Property	Primary Index	Clustering Index
Ordering field	Primary key (unique)	Non-key field (may have duplicates)
Entries	One per data block	One per distinct value
Data file order	Ordered by primary key	Ordered by clustering field
Type	Sparse	Sparse

4.2 Secondary Index

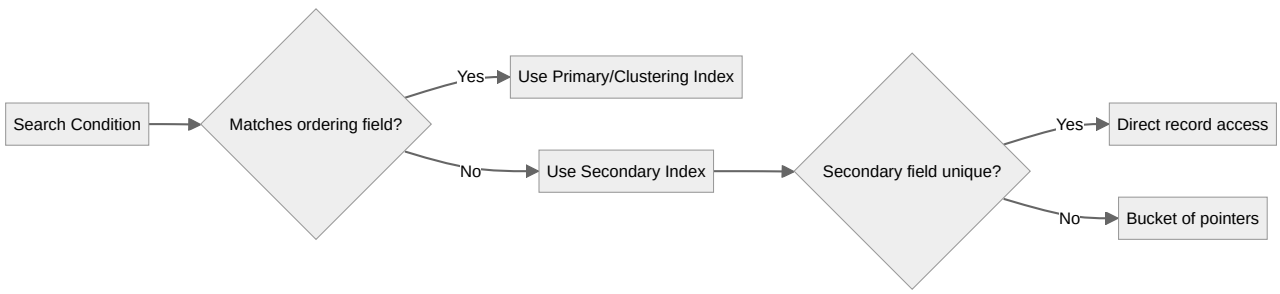
A **secondary index** provides an alternative way to access records when the primary or clustering index does not match the desired search field. The data file is **not physically ordered** by the secondary index field.

> **Secondary Index Types**
>
> **Type A: Secondary index on a key field** (unique values)
>
> - One index entry per record.
> - This is a **dense index**.
> - Entry format: <KeyValue, RecordPointer>
>
> **Type B: Secondary index on a non-key field** (duplicate values allowed)
>
> - Option 1: One index entry per record (dense) with multiple entries having the same key value.
> - Option 2: One index entry per distinct key value, where the pointer points to a bucket of record pointers.

Table 7 ***Comparison of All Index Types***

Index Type	Data File Ordered?	Dense/Sparse	Entry Points To
Primary	Yes (by primary key)	Sparse	Data block (block anchor)
Clustering	Yes (by clustering field)	Sparse	First block with that value
Secondary (key)	No	Dense	Individual record
Secondary (non-key)	No	Dense or bucket	Individual record or bucket

Important: A file can have **at most one** physical ordering, therefore at most one primary or clustering index. But it can have **many secondary indexes**.



5. Transaction Processing & Concurrency (Section 20.1)

5.1 Single-User versus Multiuser Systems

Table 8 ***Single-User vs. Multiuser DBMS***

Feature	Single-User DBMS	Multiuser DBMS
Users	At most one user at a time	Many users concurrently
Examples	Personal computer databases	Airline reservations, banks, stock exchanges
Concurrency	None	Hundreds or thousands of simultaneous transactions

How Concurrent Execution Works

Multiuser access is possible due to **multiprogramming**, allowing the Operating System to execute multiple programs (processes) at the same time.

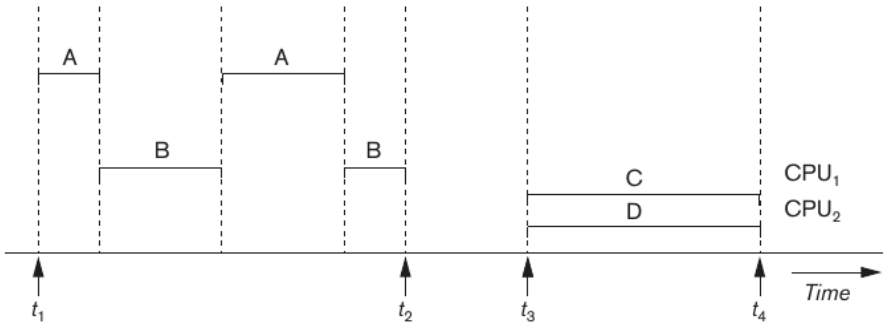


Figure 20.1
Interleaved processing versus parallel processing of concurrent transactions.

Figure 2 *Interleaved processing versus parallel processing of concurrent transactions. Left: Processes A and B take turns on CPU1. Right: Processes C and D run simultaneously on CPU1 and CPU2.*

1. Interleaved Processing (Single CPU): A single CPU can only execute one process at an exact moment. However, it executes commands from Process A, suspends A, executes Process B, suspends B, and resumes A. Interleaving keeps the CPU busy when a process is

waiting for I/O (like reading from a disk) and prevents long processes from hogging the system.

2. Parallel Processing (Multiple CPUs): If the hardware has multiple CPUs, processes can truly run at the exact same time. Process C runs on CPU1 while Process D simultaneously runs on CPU2.

> **Note**
> Most concurrency control theory in databases is based on the **interleaved model**.

5.2 Transactions, DB Items, Operations, and Buffers

> **Transaction**
> An executing program that forms a logical unit of database processing. It includes one or more database access operations (insert, delete, update, read). Transaction boundaries are defined by explicit `begin transaction` and `end transaction` statements in an application.

Types: Read-only transaction (only retrieves data) vs. Read-write transaction (updates data).

Database Items and Granularity

A database is treated as a collection of named data items. **Granularity** is the size of a data item. It could be an entire disk block, a database record, or just an individual field value (like a specific salary attribute).

Basic Access Operations

Transactions interact with these items using two basic commands:

Table 9 ***Basic Transaction Operations***

Operation	Steps
read_item(X)	1. Find the disk block address containing X. 2. Copy that disk block into a main memory buffer (if not already there). 3. Copy item X from the buffer to the program variable.
write_item(X)	1. Find the disk block address containing X. 2. Copy that disk block into a main memory buffer. 3. Copy item X from the program variable into its correct location in the buffer. 4. Store the updated disk block from the buffer back to disk. (This physically updates the database).

DBMS Buffers & Memory Management

- Step 4 of a write operation is not always immediate. The DBMS Recovery Manager and the OS maintain a cache of data buffers in main memory.
- When buffers are full and a new block needs to be loaded, the system uses a buffer replacement policy, commonly **LRU (Least Recently Used)**.
- If a buffer being replaced has been modified, it must be written back to the physical disk before the space is reused.

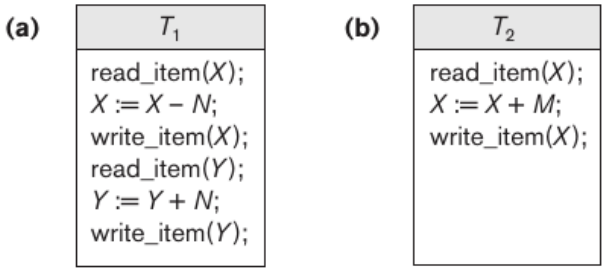


Figure 20.2
Two sample transactions.
(a) Transaction T_1.
(b) Transaction T_2.

Figure 3 *Two sample transactions. (a) Transaction T1 reads X, modifies X, writes X, reads Y, modifies Y, writes Y. (b) Transaction T2 reads X, modifies X, writes X.*

Read-Set and Write-Set

- **Read-set:** The set of all items a transaction reads.
- **Write-set:** The set of all items a transaction writes.

Example from Figure 3:

- Transaction T1 reads X, modifies X, writes X, reads Y, modifies Y, writes Y.
  Read-set = {X, Y}, Write-set = {X, Y}.
- Transaction T2 reads X, modifies X, writes X.
  Read-set = {X}, Write-set = {X}.

5.3 Why Concurrency Control Is Needed

When multiple transactions execute concurrently without strict control, disaster strikes. Let us look at four major problems using a simplified airline reservation example:

- Item X: Number of reserved seats on Flight 1.
- Item Y: Number of reserved seats on Flight 2.
- Transaction T1: Transfers N reservations from Flight X to Flight Y.
- Transaction T2: Reserves M seats on Flight X.

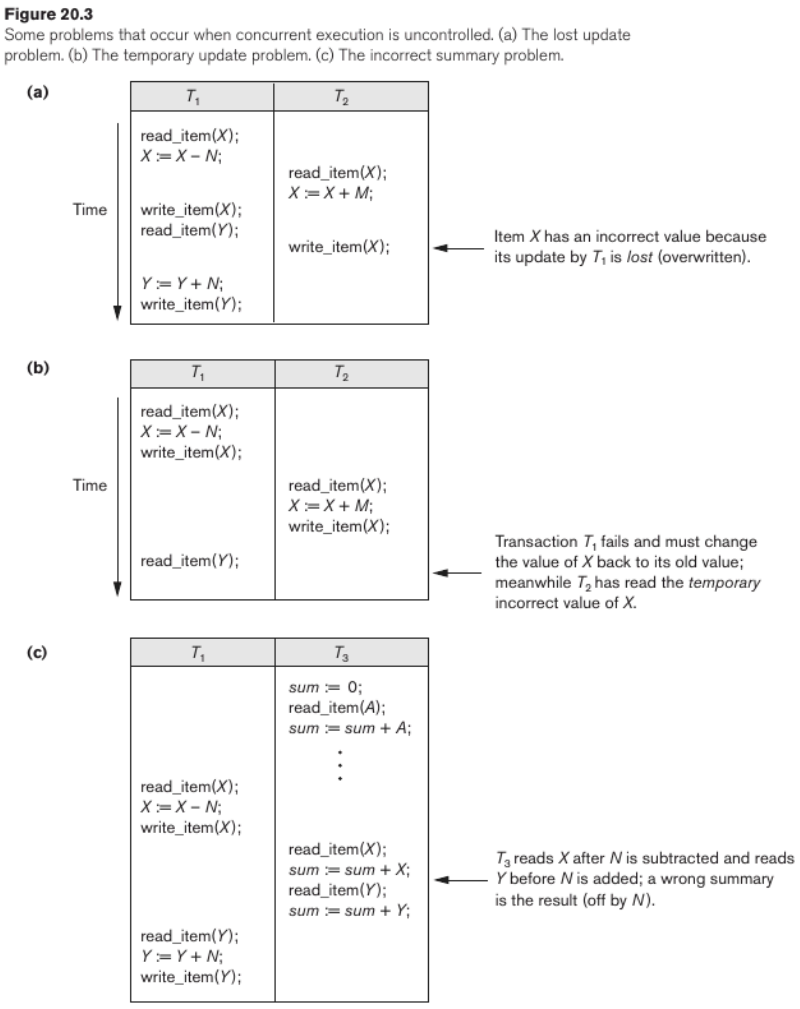


Figure 4 *Problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update (dirty read) problem. (c) The incorrect summary problem.*

> **The Scenario**
>
> - T1 and T2 are submitted at the same time.
> - T1 reads X.
> - T2 reads X.
> - T1 updates X (X := X - N) and writes X.
> - T2 updates X (X := X + M) and writes X.

The Error: Because T2 read the value of X before T1 wrote its changes, T2's final write completely overwrites (loses) T1's update.

Mathematical Proof: If initial X = 100, N = 5, M = 10. The final result should be 100 - 5 + 10 = 105. However, because T2 read X as 100 and added 10, it writes 110 to the database. T1's subtraction of 5 is lost!

2. The Temporary Update (Dirty Read) Problem (Figure 4b)

Occurs when one transaction updates an item, fails, and must be rolled back -- but another transaction reads that temporary, incorrect value before the rollback happens.

> **The Scenario**
>
> - T1 reads X, updates X (X := X - N), and writes X.
> - T2 reads X. (This is Dirty Data because T1 hasn't finished yet).
> - T1 fails for some reason! The system must roll X back to its original value.

The Error: T2 has already read the temporary, incorrect value of X and is proceeding with its calculations based on data that technically never existed permanently in the database.

3. The Incorrect Summary Problem (Figure 4c)

Occurs when one transaction calculates an aggregate summary (like a sum) while other transactions are actively updating the items being summarized.

> **The Scenario**
>
> - Transaction T3 is calculating the total number of reservations across all flights.
> - T3 reads X and adds it to sum.
> - Interleave: T1 executes completely. It subtracts N from X and adds N to Y.
> - T3 continues its loop and eventually reads Y, adding it to sum.

The Error: T3 read X after N seats were subtracted, but it read Y before those N seats were added. The final summary result will be short by exactly N seats.

4. The Unrepeatable Read Problem

Occurs when a transaction T1 reads the exact same database item twice during its execution, but another transaction T2 modifies that item in between the two reads.

> **The Scenario**
>
> A customer checks seat availability on several flights. They decide on Flight X. Before finalizing, the transaction reads Flight X's availability a second time just to be sure. Transaction T2 booked a seat on Flight X between the first and second read. The original transaction gets two completely different values for the exact same read operation.

6. Desirable Properties of Transactions (ACID) (Section 20.3)

For a database to be reliable, transactions must possess specific properties. These are commonly known as the **ACID properties**. They are enforced by the concurrency control and recovery methods of the DBMS.

6.1 The ACID Properties Explained

> **1. Atomicity (The "All or Nothing" Rule)**
>
> **Definition:** A transaction is an indivisible, atomic unit of processing. It must be performed entirely, or not performed at all.
>
> **Responsibility:** Enforced by the transaction recovery subsystem of the DBMS.
>
> **Mechanism:** If a transaction fails to complete (e.g., due to a system crash midway), the recovery technique must undo any partial effects it had on the database. If it successfully commits, its write operations must be written to disk.
>
> **Example:** You are transferring $100 from Account A to Account B. The system deducts $100 from A, but crashes before adding it to B. Because of atomicity, the system will undo the deduction from A so no money vanishes into thin air.

2. Consistency Preservation

Definition: A transaction must take the database from one valid, consistent state to another valid, consistent state. (A consistent state means the database satisfies all rules and schema constraints).

Responsibility: Enforced primarily by the programmers (who write logical database programs) and the DBMS integrity constraints module.

Example: In a bank database, a constraint states that the sum of all money across all accounts must not randomly change. If a transaction transfers money, the total sum of money in the bank must be identical before and after the transaction runs.

3. Isolation

Definition: A transaction should appear as if it is executing in complete isolation from all other transactions, even if hundreds are executing concurrently. It should experience zero interference.

Responsibility: Enforced by the concurrency control subsystem.

Mechanism: One common way to enforce this is to ensure a transaction's updates (writes) remain invisible to other transactions until it fully commits. (This entirely solves the "Dirty Read" problem mentioned earlier).

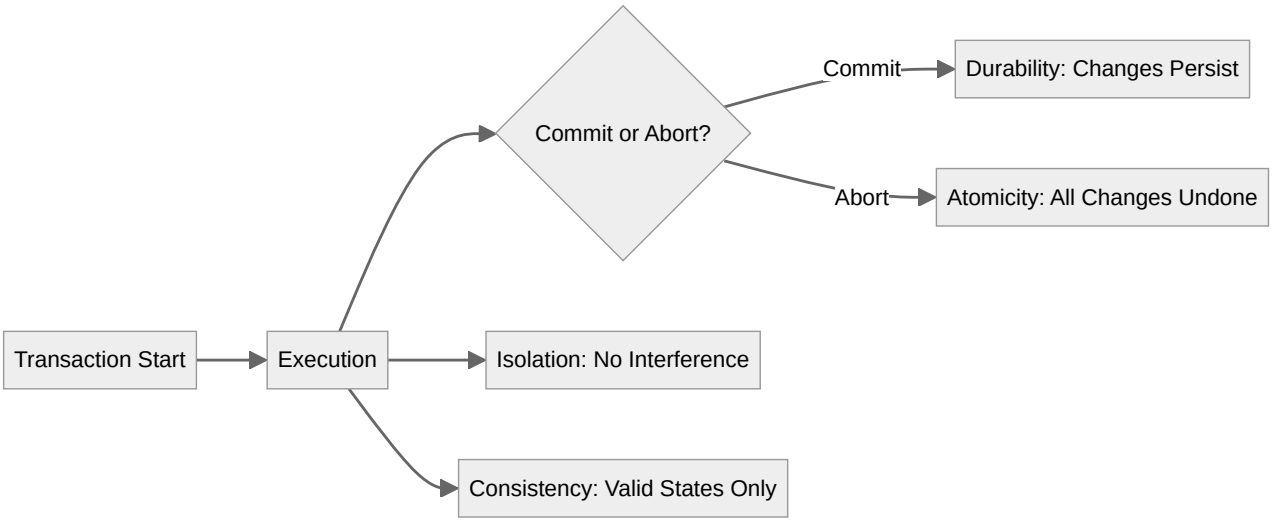
Example: If two travel agents try to book the last seat on a flight at the exact same millisecond, isolation ensures they don't see a jumbled, half-updated database. One transaction completes its full booking first, and the second one sees the seat as fully booked.

4. Durability (Permanency)

Definition: Once a transaction is "committed" (successfully completed), its changes to the database must persist permanently. They must never be lost, even in the event of a catastrophic system failure.

Responsibility: Enforced by the recovery subsystem.

Example: If you get a confirmation screen that your concert ticket is booked, and the server loses power two seconds later, your ticket is still reserved when the server boots back up.



6.2 Levels of Isolation

DBMS systems can enforce varying degrees of isolation depending on the performance vs. strictness needed. These levels build upon each other:

Table 10 ***Isolation Levels and Problems Solved***

Level	Properties	Problems Prevented
Level 0	Does not overwrite dirty reads of higher-level transactions	Basic protection
Level 1	Level 0 + no lost updates	Lost Update Problem (Figure 4a)
Level 2	Level 1 + no dirty reads	Temporary Update / Dirty Read (Figure 4b)
Level 3 (True Isolation)	Level 2 + repeatable reads	Unrepeatable Read + all above

Note: Another practical method often used in modern DBMS is called **snapshot isolation**, which is an alternative to standard locking mechanisms. In snapshot isolation, each transaction sees a consistent snapshot of the database as of the time it started.

7. Concurrency Control Techniques

This section supplements the core notes with additional concurrency control mechanisms that are essential for a complete understanding of transaction management.

7.1 Two-Phase Locking (2PL)

Two-Phase Locking is the most common concurrency control protocol. It guarantees serializability by regulating when transactions can acquire and release locks.

> **Lock Types**
>
> - **Shared Lock (S-lock / Read Lock):** A transaction can read but not write the item. Multiple transactions can hold S-locks on the same item simultaneously.
> - **Exclusive Lock (X-lock / Write Lock):** A transaction can both read and write the item. Only one transaction can hold an X-lock on an item at any time.

The Two-Phase Locking Protocol

Every transaction must follow these two phases:

1. **Growing Phase (Expanding Phase):** The transaction may obtain locks (S or X), but may not release any lock.
2. **Shrinking Phase (Contracting Phase):** The transaction may release locks, but may not obtain any new locks.

> **2PL Variants**
>
> - **Basic 2PL:** Locks are released as soon as possible after the peak (when the transaction starts shrinking).
> - **Strict 2PL:** All exclusive locks are held until the transaction commits or aborts. This prevents cascading rollbacks.
> - **Rigorous 2PL:** All locks (shared and exclusive) are held until commit/abort. Simplifies recovery but reduces concurrency.

Table 11 ***2PL Lock Compatibility Matrix***

	S-lock requested	X-lock requested
S-lock held	Compatible	Not compatible
X-lock held	Not compatible	Not compatible

7.2 Timestamp Ordering

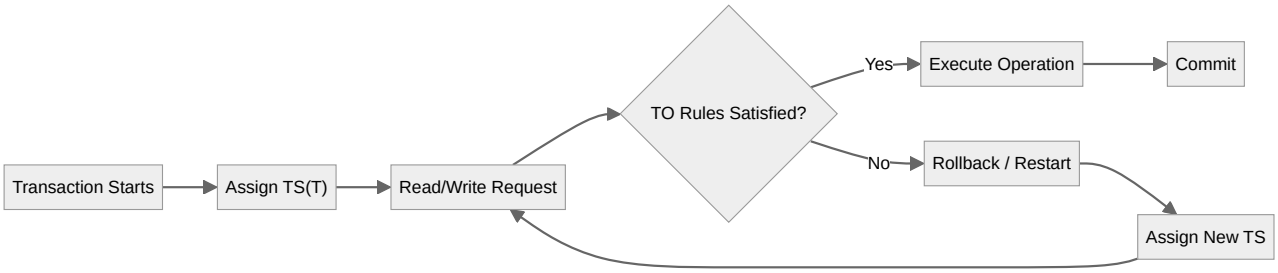
An alternative to locking that uses transaction timestamps to enforce serializability without locks or deadlock detection.

> **Timestamp Ordering Rules**
>
> Each transaction T is assigned a unique timestamp TS(T) when it starts. For each data item X, the system maintains:
>
> - **read_TS(X):** The timestamp of the youngest transaction that read X.
> - **write_TS(X):** The timestamp of the youngest transaction that wrote X.

Basic Timestamp Ordering (TO) Protocol:

- **Transaction T issues read(X):**
  - If TS(T) < write_TS(X): T is trying to read a value that was overwritten after T started. Reject the operation and roll back T (T is "too late").
  - If TS(T) ≥ write_TS(X): Allow the read. Update read_TS(X) = max(read_TS(X), TS(T)).
- **Transaction T issues write(X):**
  - If TS(T) < read_TS(X): The item was already read by a younger transaction. Reject and roll back T.
  - If TS(T) < write_TS(X): T is trying to write an obsolete value. Reject and roll back T.
  - Otherwise: Allow the write. Update write_TS(X) = TS(T).

> **Thomas Write Rule (Optimization)**
>
> If TS(T) < write_TS(X) when T tries to write X, instead of rejecting T, simply **ignore the write** (it would be obsolete anyway). This reduces unnecessary rollbacks while still guaranteeing serializability.


Transaction Starts
Assign TS(T)
Read/Write Request
TO Rules Satisfied?
Yes
Execute Operation
Commit
No
Rollback / Restart
Assign New TS

8. Recovery Techniques

Recovery ensures **atomicity** and **durability** when failures occur. The recovery manager must bring the database back to a consistent state.

8.1 Types of Failures

Table 12 ***Types of System Failures***

Failure Type	Cause	Effect
Transaction failure	Logical error, system error, deadlock	Transaction must abort
System crash	Power failure, software bug	Volatile memory lost; non-volatile storage intact
Disk failure	Head crash, hardware defect	Non-volatile storage partially lost

8.2 Log-Based Recovery

The most common recovery technique maintains a **log** (or journal) on stable storage that records all modifications to the database.

> **Log Record Types**
>
> - `[T, X, old_value, new_value]` -- Update record for transaction T on item X.
> - `[T, start]` -- Transaction T has started.
> - `[T, commit]` -- Transaction T has committed.
> - `[T, abort]` -- Transaction T has aborted.

Deferred Update (NO-UNDO/REDO)

- Updates are not written to the database until the transaction commits.
- If transaction aborts: nothing needs to be undone (NO-UNDO).

- If system crashes after commit but before disk write: REDO all committed transactions using the log.

Immediate Update (UNDO/REDO)

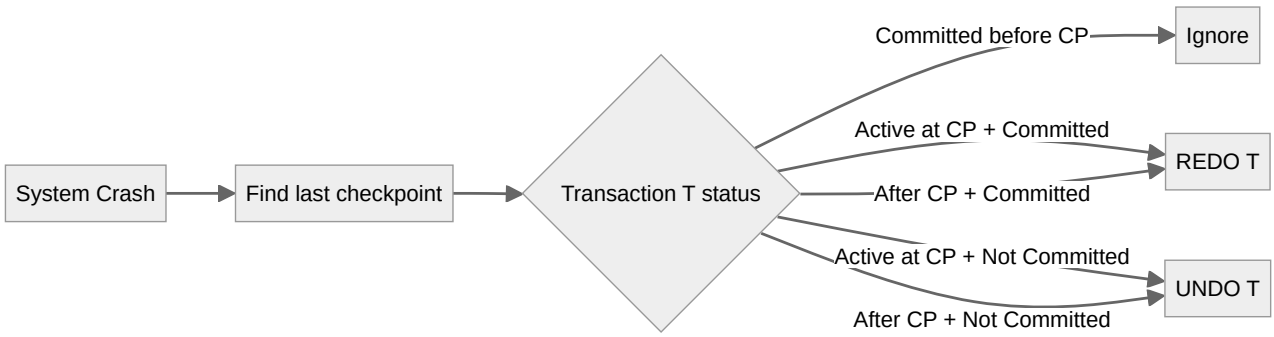
- Updates can be written to the database before commit.
- If transaction aborts: UNDO its changes using the log's old values.
- If system crashes: UNDO uncommitted transactions, REDO committed ones.

8.3 Checkpoints

To reduce recovery time, the system periodically establishes a **checkpoint**:

1. Suspend all transactions temporarily.
2. Force all modified buffers to disk.
3. Write a `[checkpoint, L]` record to the log, where L is the list of active transactions.
4. Resume transactions.

> **Checkpoint Recovery Rules**
>
> - Transactions that committed **before** the checkpoint: Already on disk, nothing to do.
> - Transactions in list L (active at checkpoint): Must be examined. If committed, REDO; if not, UNDO.
> - Transactions that started **after** the checkpoint: If committed, REDO; if not, UNDO.



9. Exam Questions & Answers

This section contains practice questions and detailed answers to help you prepare for your DBMS examination.

Chapter 1: File Operations

> **Q1. What is the difference between retrieval and update operations on files?**

A: Retrieval operations (like Find, Read) locate specific records for examination but do not change any data in the file -- they are read-only. Update operations (like Insert, Delete, Modify) change the file's contents by adding, removing, or altering records.

> **Q2. Explain the concept of a "current record" in file operations.**

A: When a search condition matches multiple records, the DBMS locates the first record based on its physical sequence on the disk. This record is designated as the current record. Subsequent search operations (like FindNext) commence from this exact position.

> **Q3. What is the difference between File Organization and Access Method?**

A: File Organization is the physical structure of how data is stored on disk (e.g., sequential ordering by Name). Access Method is the logical set of operations applied to a file (e.g., Find, Read, Insert). Several access methods can be applied to one organization, but some methods require specific organizations.

> **Q4. Describe the DBA's dilemma in file organization design.**

A: Different applications may need the same file organized differently. For example, one application searches by SSN (best order: SSN), while another groups by Department (best order: Department). Since a file can only be physically sorted one way, the DBA must choose based on the expected mix and importance of operations.

Chapter 2: Primary Indexes

Q5. Define a primary index and list its key characteristics.

A: A primary index is a supplementary, ordered file that acts as an access structure to help quickly search data records. It is an ordered file with fixed-length records. Each record has two fields: (1) a primary key matching the data file's ordering key, and (2) a block pointer to a physical disk block.

Q6. Why is a primary index always a sparse (nondense) index?

A: A primary index is always sparse because it only contains one entry per data block (the block anchor), not one entry for every single record in the file. Since a block contains multiple records, the index has fewer entries than the data file has records.

Q7. Calculate the search cost with and without a primary index given: r=30,000 records, B=1024 bytes, R=100 bytes, V=9 bytes, P=6 bytes.

A:

- Blocking factor bfr = floor(1024/100) = 10 records/block.
- Data blocks b = ceil(30000/10) = 3000 blocks.
- Without index: ceil(log2 3000) = 12 block accesses.
- Index entry size = 9 + 6 = 15 bytes.
- Index blocking factor = floor(1024/15) = 68 entries/block.
- Index blocks = ceil(3000/68) = 45 blocks.
- With index: ceil(log2 45) + 1 = 6 + 1 = 7 block accesses.

Q8. What is the major problem with primary indexes, and what are the solutions?

A: Because the main file is physically ordered, insertions and deletions are expensive. Inserting a record may require moving existing records, which changes block anchors and forces index updates. Solutions: use an unordered overflow file, use linked lists of overflow records per block, or use deletion markers (tombstones).

Chapter 3: Multilevel Indexes

> **Q9. What is fan-out, and why is it more efficient than binary search for multilevel indexes?**

A: Fan-out (fo) is the blocking factor of the index -- how many entries fit in one index block. Binary search divides the search space by 2 at each step (log2 b accesses). A multilevel index divides the search space by fo at each step (log_fo b accesses). Since fo is typically 100-500, the number of accesses is drastically reduced.

> **Q10. List four reasons why B+ trees are the dominant index structure in modern DBMSs.**

A:

1. They maintain automatic balance without expensive reorganization.
2. They support both exact-match and range queries efficiently.
3. Insertions and deletions are logarithmic time.
4. Leaf-level linking enables fast sequential access after a search.

Chapter 4: Secondary and Clustering Indexes

> **Q11. What is the difference between a clustering index and a secondary index?**

A: A clustering index is built on a non-key field where the data file is physically ordered by that field. It is sparse (one entry per distinct value). A secondary index is built on a field where the data file is NOT physically ordered by that field. It can be dense (one entry per record) or use buckets for non-key fields.

> **Q12. Can a file have multiple primary indexes? Multiple secondary indexes? Why?**

A: A file can have at most ONE primary or clustering index because the data file can only be physically ordered one way. However, it can have MANY secondary indexes because secondary indexes are separate ordered files that do not affect the physical ordering of the data file.

Chapter 5: Concurrency Problems

Q13. Describe the Lost Update Problem and how it occurs.

A: The Lost Update Problem occurs when two transactions read the same item, then one writes before the other reads, and the second write overwrites the first. Example: T1 and T2 both read X=100. T1 sets X=95 and writes. T2 (still using X=100) sets X=110 and writes. T1's update to 95 is lost.

Q14. What is a Dirty Read, and why is it dangerous?

A: A Dirty Read occurs when transaction T2 reads an item that has been modified by transaction T1, but T1 has not yet committed. If T1 later aborts and rolls back its changes, T2 has used data that never permanently existed in the database, leading to incorrect decisions or writes.

Q15. Explain the Incorrect Summary Problem.

A: This occurs when a transaction calculating an aggregate (like a sum) reads some items before another transaction updates them and other items after the update. The aggregate reflects a partial, inconsistent state. Example: T3 sums X+Y. It reads X before T1 subtracts N, but reads Y after T1 adds N. The sum is off by N.

Chapter 6: ACID Properties

Q16. Name and define the four ACID properties of transactions.

A:

- **Atomicity:** All operations complete successfully, or none do. Enforced by recovery.
- **Consistency:** Database moves from one valid state to another. Enforced by integrity constraints and correct programs.
- **Isolation:** Each transaction executes as if alone, despite concurrency. Enforced by concurrency control.
- **Durability:** Committed changes survive any failure. Enforced by recovery.

Q17. Which isolation level solves which concurrency problem?

A:

- Level 1: Prevents lost updates.
- Level 2: Prevents lost updates and dirty reads.
- Level 3: Prevents lost updates, dirty reads, and unrepeatable reads (true isolation).

Chapter 7: Concurrency Control Techniques

Q18. Explain the Two-Phase Locking protocol and its phases.

A: In 2PL, every transaction has two phases: (1) Growing Phase -- the transaction may obtain locks but may not release any. (2) Shrinking Phase -- the transaction may release locks but may not obtain any new locks. This protocol guarantees serializability.

Q19. What is the difference between Strict 2PL and Basic 2PL?

A: In Basic 2PL, locks can be released as soon as the shrinking phase begins. In Strict 2PL, all exclusive (write) locks are held until the transaction commits or aborts. Strict 2PL prevents cascading rollbacks but reduces concurrency compared to Basic 2PL.

Q20. How does Timestamp Ordering prevent serializability violations without using locks?

A: Each transaction gets a unique timestamp. For each data item, the system tracks read_TS and write_TS. If a transaction tries to read an item written by a younger transaction, or write an item already read/written by a younger transaction, it is rolled back and restarted with a new timestamp. This enforces an order equivalent to serial execution by timestamp.

Chapter 8: Recovery

Q21. What is the purpose of a checkpoint in log-based recovery?

A: A checkpoint reduces the amount of log that must be examined during recovery. At a checkpoint, the system forces all modified buffers to disk and writes a checkpoint record listing active transactions. During recovery, transactions that committed before the checkpoint need no action, limiting REDO/UNDO work to recent transactions.

Q22. In immediate update (UNDO/REDO) recovery, what actions are taken after a crash?

A: After a crash with immediate update, the recovery manager: (1) REDOs all committed transactions to ensure their updates are on disk (durability). (2) UNDOs all uncommitted (active) transactions to erase any partial changes they wrote before the crash (atomicity).

Comprehensive / Long-Answer Questions

> **Q23. Compare and contrast Primary, Clustering, and Secondary indexes with respect to: (a) whether the data file is ordered, (b) dense vs. sparse, (c) what the pointer points to, and (d) maximum number allowed per file.**

A:

Index	Data Ordered?	Type	Pointer	Max per File
Primary	Yes, by primary key	Sparse	Data block	1
Clustering	Yes, by clustering field	Sparse	First block with value	1
Secondary (key)	No	Dense	Individual record	Many
Secondary (non-key)	No	Dense or bucket	Record or bucket	Many

> **Q24. With a diagram or clear step-by-step explanation, show how Two-Phase Locking prevents the Lost Update Problem.**

A: Consider T1 and T2 both wanting to write X.

1. T1 requests and acquires X-lock on X (growing phase).
2. T2 requests X-lock on X but must wait because T1 holds it.
3. T1 reads X, modifies X, writes X, then commits and releases X-lock (shrinking phase).
4. T2 now acquires X-lock, reads the updated X, modifies it, and writes.

Because T2 was blocked until T1 finished, T2 reads the value after T1's update. No update is lost.

> **Q25. Explain how the ACID properties map to the concurrency and recovery subsystems of a DBMS.**

A:

- **Atomicity:** Enforced by the recovery subsystem. Uses logs to UNDO partial changes of failed transactions.
- **Consistency:** Enforced by programmers writing correct logic and by the DBMS integrity constraints module (keys, checks, triggers).
- **Isolation:** Enforced by the concurrency control subsystem. Uses locking (2PL) or timestamp ordering to prevent interleaving conflicts.
- **Durability:** Enforced by the recovery subsystem. Uses force-writing logs and checkpoints to ensure committed data survives crashes.